AI TRANSIT PIPELINE

Securing AI Code Integration in the Enterprise

- Secure retrieval from GitHub
- Adaptive multi-layer scan by file type
- Approved ZIP archive + Excel traceability report

Table of Contents

01 Context & Objectives

Why this pipeline?

- AI-generated code may contain backdoors, secrets, or dangerous patterns
- Developers import code without systematic security review
- The corporate network must remain isolated (partial air-gap)
- No traceability without a structured process

Key Objectives

Retrieve

Secure depth-1 clone

Scan

Adaptive multi-layer

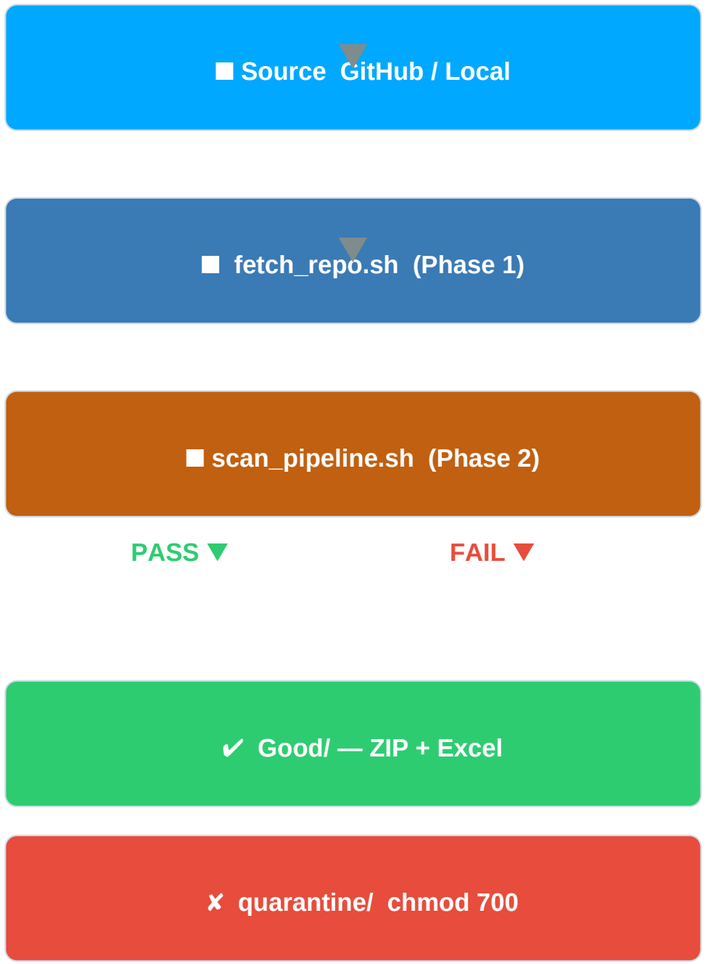
Decide

PASS→ZIP / FAIL→Quarantine

Trace

Excel + JSON + HTML

02 Pipeline Architecture



Phase 1 — fetch_repo.sh

- Host whitelist (github.com only)
- Size check via GitHub API (< 500 MB)
- git clone --depth 1 --no-tags
- .git/ removal (no metadata)
- SHA-256 manifest of all files
- Quick scan for suspicious patterns

Phase 2 — scan_pipeline.sh

■ GLOBAL Layer:

Gitleaks · detect-secrets · ClamAV · YARA

■ PER-TYPE Layer:

.py → Bandit + eval/exec

.js/.ts → Semgrep + child_process

.sh → ShellCheck + curl|bash

.yml → unpinned actions

.tf → checkov + hardcoded secrets

Dockerfile → hadolint + :latest

.so/.exe → auto FAIL + strings

03 Phase 1 — Secure Retrieval

1 Host Whitelist

Only github.com is allowed.

Any other URL is rejected immediately.

2 Size Verification

GitHub API queried before clone.

Limit: 500 MB.

3 Minimal Clone

git clone --depth 1 --no-tags

--single-branch

4 .git/ Removal

Git metadata removed

(hooks, remotes, submodules).

5 SHA-256 Manifest

Hash of each file →

.manifest_sha256.txt (audit trail).

6 Quick Triage

Grep: eval(exec(curl|bash

rm -rf → immediate alert.

04 Phase 2 — Global Layer

Applies to the ENTIRE directory, regardless of file type

■ Gitleaks

- > 150 credential types
- GitHub/AWS/GCP/Azure tokens
- RSA, PEM, SSH, JWT keys
- Entropy analysis + regex

FAIL if positive

■ detect-secrets

- Shannon entropy
- Secrets unknown to rule sets
- High-density base64/hex
- Complements Gitleaks

FAIL if positive

■ ClamAV

- Millions of signatures
- Trojans, ransomware, backdoors
- Full recursive scan
- freshclam updated base

FAIL if positive

■ YARA

- Industry-standard IOC
- Custom .yar rules
- AI backdoor patterns
- Regex + boolean logic

FAIL if positive

04 Phase 2 — Dispatch by File Type

Extension(s)	SAST Tool	Manual Checks	Verdict
.py	Bandit	eval() exec() import os/subprocess	FAIL
.js .ts .jsx	Semgrep	eval() child_process require(…)	FAIL
.c .cpp .h	cppcheck	gets() strcpy() system() popen()	FAIL
.sh .bash .ps1	ShellCheck	curl|bash wget|sh eval $var	FAIL
.yml .yaml	—	Unpinned actions inline secrets	FAIL
.tf .tfvars	checkov	Hardcoded secrets overly broad IAM	FAIL
Dockerfile	hadolint	:latest ADD http:// RUN curl|bash	FAIL
.so .exe .elf	strings	Auto FAIL + IOC strings	FAIL ■
.zip .tar.gz	—	Auto FAIL — re-scan after extraction	FAIL ■
.sql	—	xp_cmdshell DROP TABLE ; --	FAIL
.json .md .txt	—	password: secret: token: api_key=	FAIL

05 Per-type Check Details

Python .py

Dynamic execution — high risk of backdoors via eval/exec

Bandit SAST

FAIL

Python AST: subprocess shell=True, pickle, MD5, SQL, assert…

eval() / exec()

FAIL

Arbitrary execution at runtime — trivial backdoor vector

import os/subprocess

WARN

Shell access — requires manual review

JavaScript .js .ts

Ecosystem exposed to supply-chain attacks

Semgrep p/javascript

FAIL

XSS, prototype pollution, innerHTML, SSTI…

eval()

FAIL

Dynamic execution — classic injection vector

child_process

FAIL

Shell command execution from Node.js

05 Per-type Check Details

C / C++ .c .cpp

Risks from manual memory management

cppcheck

FAIL

Buffer overflows, null ptr, use-after-free, uninit memory.

gets() strcpy()

FAIL

Unbounded read/copy → buffer overflow

system() popen()

FAIL

Shell command execution → possible injection

Shell .sh .bash

Direct execution — command obfuscation is trivial

ShellCheck

FAIL

Unquoted variables, globbing, dangerous redirections…

curl | bash

FAIL

Remote code execution without integrity check

eval $variable

FAIL

Trivial injection if variable controlled by attacker

05 Per-type Check Details

YAML / CI .yml

CI/CD pipelines — a bad config opens backdoors

Unpinned actions

FAIL

uses: action@main → vulnerable to tag-hijacking

Inline secrets

FAIL

password: value (not ${{ secrets.XXX }})

Terraform .tf

IaC — can provision entire cloud resources

checkov (CIS)

FAIL

S3 buckets, overly broad IAM, DB without at-rest encryption

Hardcoded secrets

FAIL

password = "value" in .tf/.tfvars

05 Checks by Type — Dockerfile · Binaries · Archives · SQL

Dockerfile

- **hadolint** — FAIL

  CIS Docker: :latest, ADD http://, apt without --no-install-recommends

- **RUN curl|bash** — FAIL

  Download + execution without integrity control

Binaries .so .exe .elf

- **Auto FAIL** — FAIL

  No binaries in an AI repo — potential implant/RAT

- **strings + IOC** — FAIL

  /bin/sh /etc/passwd exec reverse shell URLs detected

Archives .zip .tar.gz

- **Auto FAIL** — FAIL

  Content cannot be scanned without prior extraction

- **Zip-slip** — FAIL

  Extraction to arbitrary paths (../../etc/cron.d/)

SQL .sql

- **xp_cmdshell** — FAIL

  Shell command execution from SQL Server → critical IOC

- **DROP TABLE / --** — FAIL

  Destructive statements + SQL injection pattern

06 Tools Used — Summary

All optional in degraded mode — WARN if absent, FAIL only on active finding

Tool	Role	Installation	Scope
Gitleaks	secrets/tokens	GitHub binary	Global
detect-secrets	high entropy	pip install	Global
ClamAV	malware signatures	apt install clamav	Global
YARA	custom IOC	apt install yara	Global
Bandit	Python SAST	pip install bandit	.py
Semgrep	multi-lang SAST	pip install semgrep	.js .ts
cppcheck	C/C++ static analysis	apt install cppcheck	.c .cpp
ShellCheck	shell linting	apt install shellcheck	.sh .bash
hadolint	Dockerfile linting	GitHub binary	Dockerfile
checkov	IaC security	pip install checkov	.tf .hcl
jq	GitHub API JSON parsing	apt install jq	Utility

07 Results — ZIP Archive & Excel Report

■ ZIP Archive → Good/

```
Good/
■■■ repo_20260615_143022.zip
    ■■■ src/
    ■   ■■■ [full source code]
    ■■■ README.md
    ■■■ scan_report_20260615.xlsx
```

- ✔ Produced only if verdict is PASS
- ✔ Timestamped name — guaranteed uniqueness
- ✔ .manifest_sha256.txt excluded
- ✔ Excel report included directly
- ✔ Ready for internal network transfer
- ✔ Optional: GPG encryption

■ Excel Report (.xlsx)

Tab 0 — Summary

Repository / Source	GitHub URL or local path
Scan Date	ISO 8601 timestamp
SHA-256 Hash	Global fingerprint of all files
Verdict	PASS (green) / FAIL (red)
Counters	PASS · WARN · FAIL

Tab 1 — Files (one row per file)

#	File	Type	Status	Message
1	src/main.py	.py	FAIL	bandit:HIGH
2	config.sh	.sh	WARN	shellcheck absent
3	README.md	.md	PASS	—

Sorted FAIL→WARN→PASS | Auto-filter | Row 1 frozen

08 Absolute Security Rules

■ **These rules CANNOT be modified — they constitute the threat model**

■ Network Isolation

The transit machine NEVER has direct access to the corporate network.

➡ One-way Flow

Only approved/ is readable from the inside — not fetch/, logs/, quarantine/.

■ Root-only Quarantine

chmod 700 — no application user can read rejected files.

■ No Manual Move

Never quarantine/ → approved/ without a full pipeline re-scan.

■ Binaries = Absolute FAIL

No .so/.exe/.elf in an AI repo — zero exceptions.

■ Archives = Mandatory Re-scan

.zip/.tar.gz → extraction + re-scan in a dedicated isolated environment.

Summary

Source

GitHub (whitelist)

or local path

Fetch

Minimal clone

.git removed · SHA-256

Scan

Global layer

+ 11 file types

PASS

ZIP in Good/

with Excel included

FAIL

Quarantine

chmod 700 + report

Bash pipeline · Degraded mode · Optional tools · Timestamped reports

github.com/Gaillotte/Claude — branch claude/vigilant-carson-f8twy0